

WILDLIFE CONSERVATION MONITORING DATABASE MANAGEMENT SYSTEM



Student Name: Darius Isaack Mandara

Student ID: 22118297

Contribution: Database design, backend development,
frontend UI design, SQL queries, testing, deployment

1. Introduction

1.1 Overview

The Wildlife Conservation Monitoring Database Management System is a web-based application developed to assist wildlife conservation organizations in managing and monitoring wildlife data efficiently. The system provides a centralized platform for storing, updating, and retrieving information related to animals, habitats, staff members, wildlife observations, and incidents.

The system was developed using Flask (Python) for the backend, PostgreSQL as the database management system, and HTML/CSS/JavaScript for the graphical user interface. The database is hosted on a cloud platform using Supabase.

The application allows authorized users to:

- Log into the system securely
- Manage animal records
- Manage habitat information
- Record wildlife observations
- Monitor wildlife incidents
- Generate analytical reports and statistics
- Visualize conservation data using charts and tables

The purpose of the system is to improve wildlife monitoring processes, data organization, and reporting efficiency within conservation environments.

1.2 Objectives of the System

The main objectives of the system are:

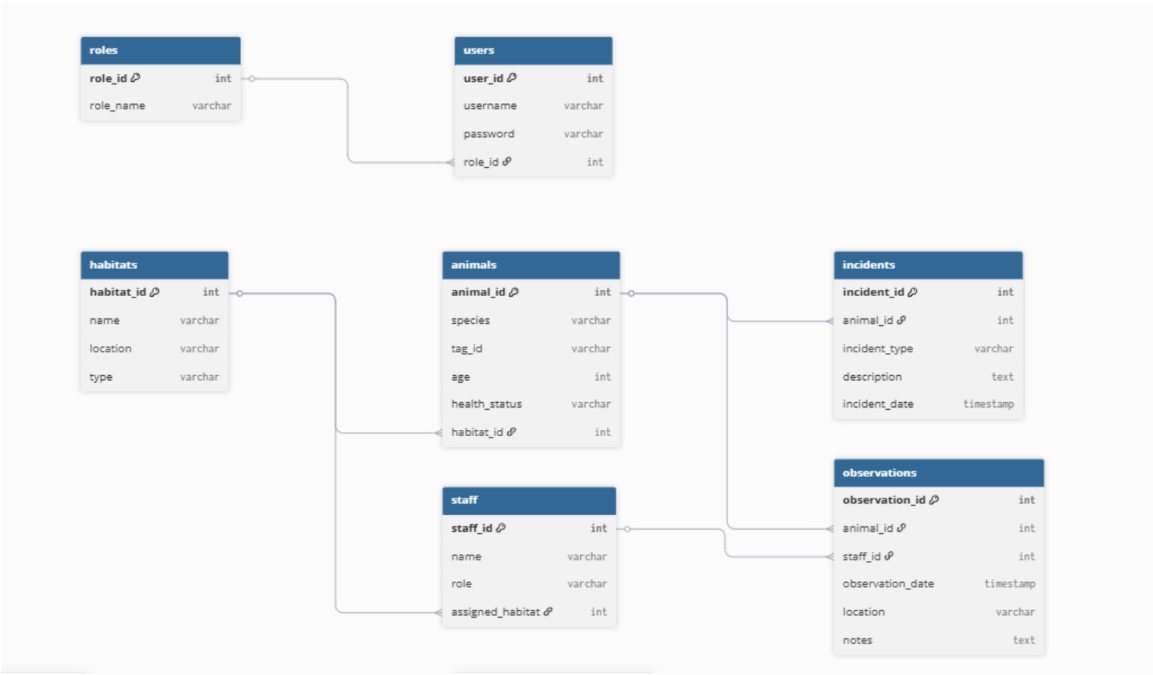
- To provide a centralized wildlife database system
- To simplify data entry and management
- To improve tracking of wildlife observations and incidents
- To generate useful analytical reports for decision-making
- To demonstrate database relationships and SQL functionality
- To implement CRUD operations using a GUI application

1.3 Technologies Used

Component	Technology
Backend	Python Flask
Frontend	HTML, CSS, JavaScript
Database	PostgreSQL
Cloud Database	Supabase
Charts	Chart.js
Development Environment	Visual Studio Code

2. Database

2.1 Entity Relationship Diagram (ERD)



The database was designed using relational database principles. The system contains seven interconnected tables that store wildlife conservation data.

Main Entities

1. Roles
2. Users
3. Habitats
4. Animals
5. Staff
6. Observations
7. Incidents

Relationships

- One role can belong to many users
- One habitat can contain many animals
- One habitat can have many staff members
- One animal can have many observations
- One staff member can record many observations
- One animal can have many incidents

2.2 Data Definition Language (DDL)

Roles Table

```
CREATE TABLE roles (  
  role_id SERIAL PRIMARY KEY,  
  role_name VARCHAR(50) UNIQUE NOT NULL  
);
```

Users Table

```
CREATE TABLE users (  
  user_id SERIAL PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,
```

```
password VARCHAR(100) NOT NULL,  
role_id INT,  
FOREIGN KEY (role_id) REFERENCES roles(role_id)  
ON DELETE SET NULL  
);
```

Habitats Table

```
CREATE TABLE habitats (  
habitat_id SERIAL PRIMARY KEY,  
name VARCHAR(100) NOT NULL,  
location VARCHAR(100),  
type VARCHAR(50)  
);
```

Animals Table

```
CREATE TABLE animals (  
animal_id SERIAL PRIMARY KEY,  
species VARCHAR(100) NOT NULL,  
tag_id VARCHAR(50) UNIQUE,  
age INT CHECK (age >= 0),  
health_status VARCHAR(50),  
habitat_id INT,  
FOREIGN KEY (habitat_id) REFERENCES habitats(habitat_id)  
ON DELETE SET NULL  
);
```

Staff Table

```
CREATE TABLE staff (  
  staff_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  role VARCHAR(50),  
  assigned_habitat INT,  
  FOREIGN KEY (assigned_habitat) REFERENCES habitats(habitat_id)  
  ON DELETE SET NULL  
);
```

Observations Table

```
CREATE TABLE observations (  
  observation_id SERIAL PRIMARY KEY,  
  animal_id INT,  
  staff_id INT,  
  observation_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  location VARCHAR(100),  
  notes TEXT,  
  FOREIGN KEY (animal_id) REFERENCES animals(animal_id)  
  ON DELETE CASCADE,  
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id)  
  ON DELETE SET NULL  
);
```

Incidents Table

```
CREATE TABLE incidents (  
  incident_id SERIAL PRIMARY KEY,  
  animal_id INT,  
  incident_type VARCHAR(50),  
  description TEXT,  
  incident_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (animal_id)  
  REFERENCES animals(animal_id)  
  ON DELETE CASCADE  
);
```

2.3 Data Manipulation Language (DML)

Sample Data for Roles

```
INSERT INTO roles (role_name)  
VALUES  
('Admin'),  
('Ranger'),  
('Researcher');
```

Sample Data for Users

```
INSERT INTO users (username, password, role_id)  
VALUES  
('admin1', 'admin123', 1),  
('ranger1', 'ranger123', 2),  
('researcher1', 'research123', 3);
```

Sample Data for Habitats

```
INSERT INTO habitats (name, location, type)
VALUES
('Savannah Zone', 'Kenya', 'Grassland'),
('Rainforest Zone', 'Amazon', 'Forest'),
('Mountain Reserve', 'Nepal', 'Mountain');
```

Sample Data for Animals

```
INSERT INTO animals (species, tag_id, age, health_status, habitat_id)
VALUES
('Elephant', 'E001', 25, 'Healthy', 1),
('Lion', 'L001', 10, 'Healthy', 1),
('Tiger', 'T001', 8, 'Injured', 2),
('Panda', 'P001', 6, 'Healthy', 3);
```

Sample Data for Staff

```
INSERT INTO staff (name, role, assigned_habitat)
VALUES
('John Doe', 'Ranger', 1),
('Jane Smith', 'Researcher', 2),
('Mike Johnson', 'Ranger', 3);
```

Sample Data for Observations

```
INSERT INTO observations (animal_id, staff_id, location, notes)
VALUES
(1, 1, 'North Savannah', 'Animal grazing normally'),
(2, 1, 'East Savannah', 'Lion resting under tree'),
```



```
(3, 2, 'Deep Forest', 'Tiger limping slightly'),  
(4, 3, 'High Mountain', 'Panda feeding');
```

Sample Data for Incidents

```
INSERT INTO incidents (  
    animal_id,  
    incident_type,  
    description  
)  
VALUES  
(1, 'Injury', 'Elephant injured near river'),  
(2, 'Poaching Attempt', 'Suspicious activity detected');
```

3. Queries

3.1 Animals Per Habitat

SQL Query

```
SELECT h.name, COUNT(a.animal_id)  
FROM habitats h  
LEFT JOIN animals a  
ON h.habitat_id = a.habitat_id  
GROUP BY h.name;
```

Purpose

This query displays the number of animals assigned to each habitat.

Output

Results	Explain	Chart	Export ▾
name		count	
Rainforest Zone		1	
Savannah Zone		4	
River Area		0	
Mountain Reserve		1	

3.2 Average Age Per Species

SQL Query

SELECT species, AVG(age)

FROM animals

GROUP BY species;

Purpose

This query calculates the average age of each animal species.

Output

Results	Explain	Chart	Export ▾
species	round		
Lion	10.00		
Panda	6.00		
Zebra	4.50		
Tiger	8.00		
Elephant	25.00		

3.3 Observation Count Per Staff

SQL Query

```
SELECT s.name, COUNT(o.observation_id)
FROM staff s
LEFT JOIN observations o
ON s.staff_id = o.staff_id
GROUP BY s.name;
```

Purpose

This query displays how many observations each staff member has recorded.

Output

Results		Explain	Chart	Export ▾
name	count			
Mike Johnson	1			
John Doe	2			
Jane Smith	1			

3.4 Animals With Incidents

SQL Query

```
SELECT a.species, i.incident_type
FROM animals a
JOIN incidents i
ON a.animal_id = i.animal_id;
```

Purpose

This query displays animals involved in incidents.

Output

Results

Explain

Chart

Export

▼

species	incident_type
Elephant	Injury
Lion	Poaching Attempt

3.5 Recent Observations

SQL Query

```
SELECT *
FROM observations
ORDER BY observation_date DESC;
```

Purpose

This query displays the most recent wildlife observations.

Output

Results

Explain

Chart

Export

▼

observation_i	animal_id	staff_id	observation_date	location	notes
1	1	1	2026-05-05 17:47:39.756137	North Savannah	Animal grazing normally
2	2	1	2026-05-05 17:47:39.756137	East Savannah	Lion resting under tree
3	3	2	2026-05-05 17:47:39.756137	Deep Forest	Tiger limping slightly
4	4	3	2026-05-05 17:47:39.756137	High Mountain	Panda feeding

4. Stored Procedures, Functions, and Triggers

4.1 Function: Count Animals in Habitat

```
CREATE OR REPLACE FUNCTION count_animals_in_habitat(h_id INT)
RETURNS INT AS
$$
DECLARE
total INT;
BEGIN
SELECT COUNT(*)
INTO total
FROM animals
WHERE habitat_id = h_id;

RETURN total;
END;
$$ LANGUAGE plpgsql;
```

4.2 Procedure: Add New Animal

```
CREATE OR REPLACE PROCEDURE add_new_animal(
p_species VARCHAR,
p_tag_id VARCHAR,
p_age INT,
p_health VARCHAR,
p_habitat INT
)
LANGUAGE plpgsql
AS
```

\$\$

BEGIN

INSERT INTO animals (species, tag_id, age, health_status, habitat_id)

VALUES (p_species, p_tag_id, p_age, p_health, p_habitat);

END;

\$\$;

4.3 Trigger: Prevent Negative Animal Age

CREATE OR REPLACE FUNCTION check_animal_age()

RETURNS TRIGGER AS

\$\$

BEGIN

IF NEW.age < 0 THEN

RAISE EXCEPTION 'Age cannot be negative';

END IF;

RETURN NEW;

END;

\$\$ LANGUAGE plpgsql;

CREATE TRIGGER animal_age_trigger

BEFORE INSERT OR UPDATE

ON animals

FOR EACH ROW

EXECUTE FUNCTION check_animal_age();

4.4 Trigger: Automatic Incident Date

CREATE OR REPLACE FUNCTION auto_incident_date()

RETURNS TRIGGER AS

```
$$  
BEGIN  
NEW.incident_date = CURRENT_TIMESTAMP;  
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
CREATE TRIGGER incident_date_trigger  
BEFORE INSERT  
ON incidents  
FOR EACH ROW  
EXECUTE FUNCTION auto_incident_date();
```

4.5 Function: Count Staff Observations

```
CREATE OR REPLACE FUNCTION count_staff_observations(s_id INT)  
RETURNS INT AS  
$$  
DECLARE  
total INT;  
BEGIN  
SELECT COUNT(*)  
INTO total  
FROM observations  
WHERE staff_id = s_id;  
RETURN total;  
END;  
$$ LANGUAGE plpgsql;
```

5. Software Implementation

5.1 Backend Development

The backend of the system was developed using Flask, a lightweight Python web framework. Flask was used to create routes, connect to the PostgreSQL database, and process user requests.

The backend handles:

- User authentication
- Database CRUD operations
- Report generation
- Data retrieval
- Dynamic rendering using Jinja2 templates

5.2 Frontend Development

The frontend was developed using:

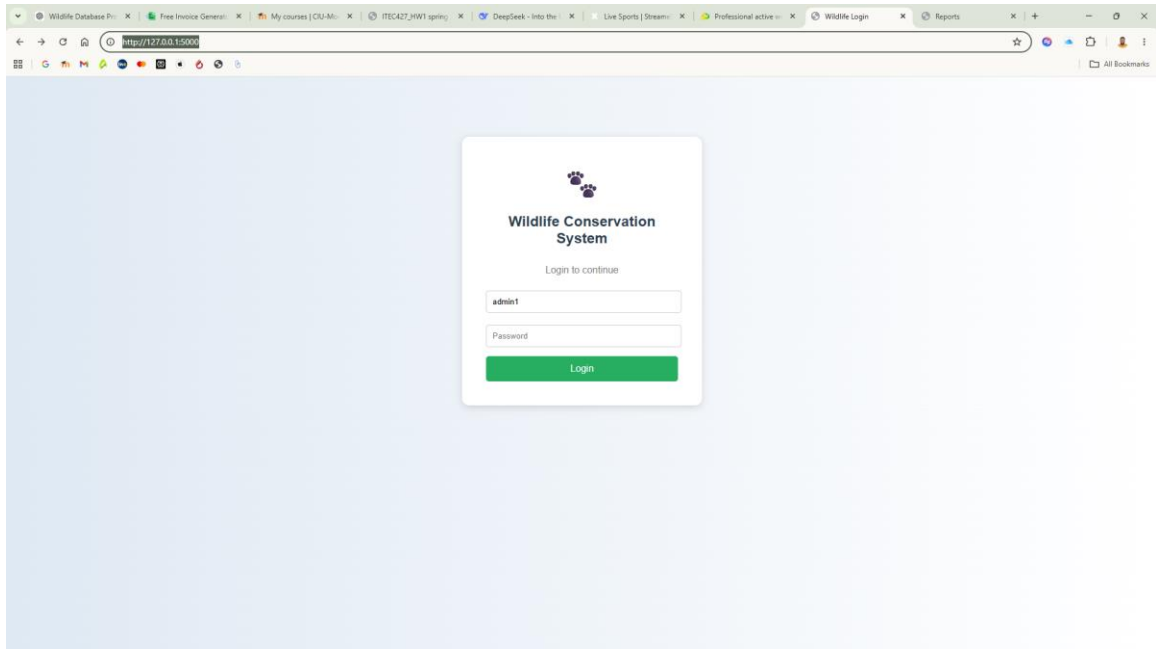
- HTML
- CSS
- JavaScript
- Chart.js

The user interface provides:

- Dashboard navigation
- Forms for data entry
- Report visualizations
- Data tables
- Analytics charts

5.3 System Screens

Login Page



The screenshot shows a web browser window with the URL `http://127.0.0.1:5000`. The page displays the "Wildlife Conservation System" login interface. It features a central white card with a paw print icon, the system name, and a "Login to continue" prompt. Below this are input fields for "admin1" (username) and "Password", followed by a green "Login" button.

Wildlife Conservation System

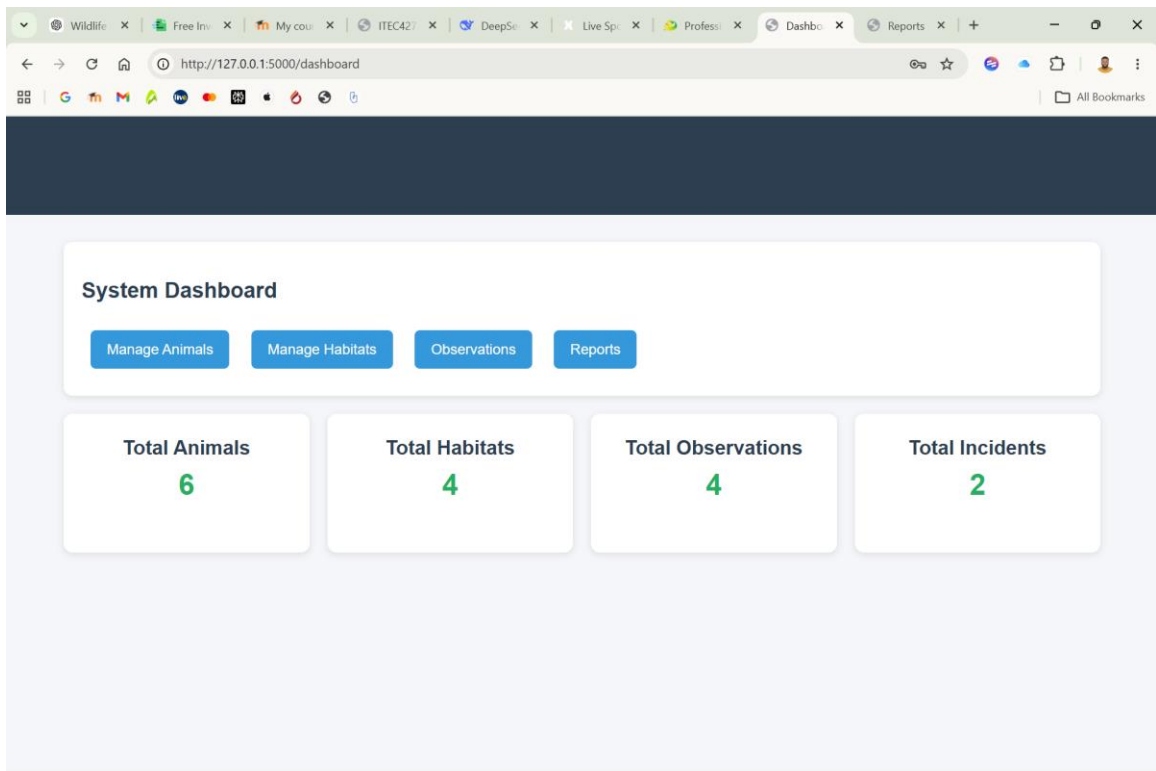
Login to continue

admin1

Password

Login

Dashboard



Animal Management Page

Wildlife XMy cou XITEC42 XDeepSe XLive Sp XProfess XAnimals XReports X+

http://127.0.0.1:5000/animals

☆🌐🔖👤⋮

📁 All Bookmarks

Tag ID

Age

Health Status

Habitat ID

Add Animal

ID	Species	Tag ID	Age	Health	Habitat	Action
1	Elephant	E001	25	Healthy	1	Edit Delete
2	Lion	L001	10	Healthy	1	Edit Delete
3	Tiger	T001	8	Injured	2	Edit Delete
4	Panda	P001	6	Healthy	3	Edit Delete
5	Zebra	5	4	Injured	1	Edit Delete
6	Zebra	Z001	5	Healthy	1	Edit Delete

[Back to Dashboard](#)

Habitat Management Page

Wildlife xFree Ins xMy cour xITEC427 xDeepSe xLive Sp xProfessi xHabitat xReports x+

←→↺🏠🔍http://127.0.0.1:5000/habitats☆🌐☁️🔖👤⋮

🗃️🌐🔍📧📺🍷🍎🔥🔄🔗

All Bookmarks

Habitat Name

Location

Type

Add Habitat

ID	Name	Location	Type	Action
1	Savannah Zone	Kenya	Grassland	Edit Delete
2	Rainforest Zone	Amazon	Forest	Edit Delete
3	Mountain Reserve	Nepal	Mountain	Edit Delete
4	River Area	Tanzania	Desert	Edit Delete

[Back to Dashboard](#)

Observation Management Page

Wildlife xFree Inv xMy cou xITEC42 xDeepSe xLive Sp xProfessi xObserv xReports x+ - o x

http://127.0.0.1:5000/observations

Animal ID

Staff ID

Location

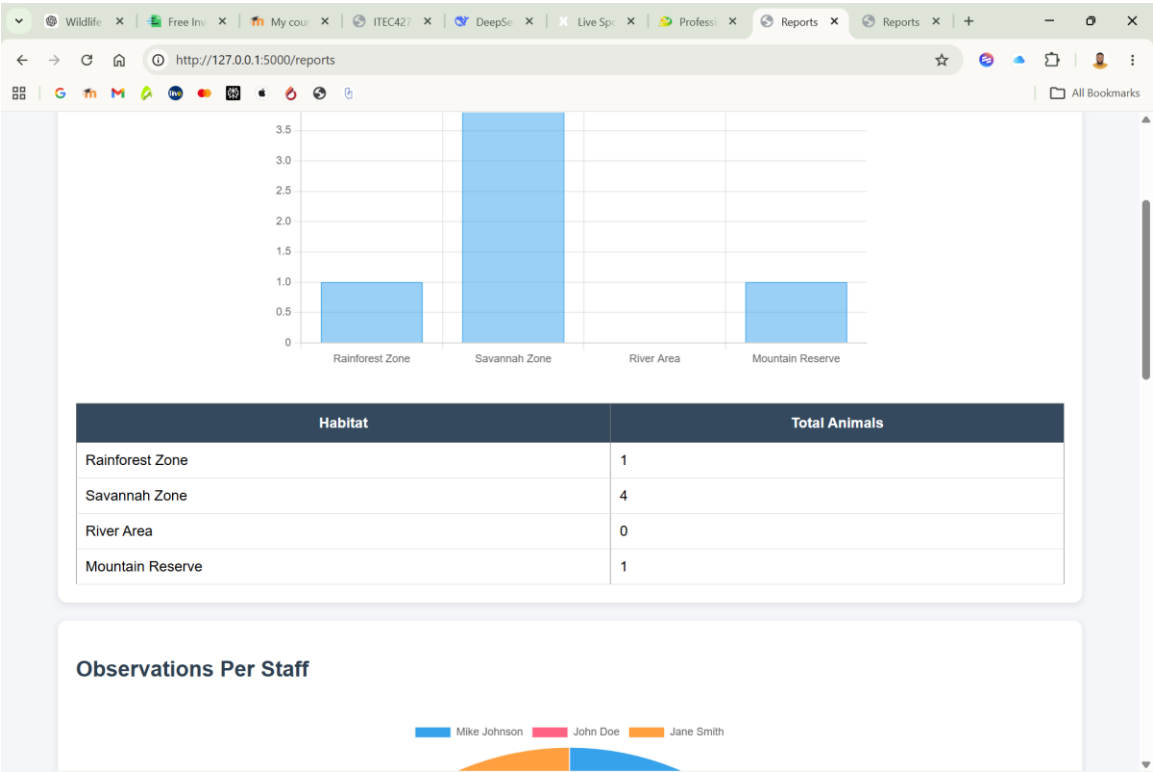
Observation Notes

Add Observation

ID	Animal	Staff	Location	Notes	Date	Action
1	Elephant	John Doe	North Savannah	Animal grazing normally	2026-05-05 17:47:39.756137	Delete
2	Lion	John Doe	East Savannah	Lion resting under tree	2026-05-05 17:47:39.756137	Delete
3	Tiger	Jane Smith	Deep Forest	Tiger limping slightly	2026-05-05 17:47:39.756137	Delete
4	Panda	Mike Johnson	High Mountain	Panda feeding	2026-05-05 17:47:39.756137	Delete

Back to Dashboard

Reports and Analytics Page



6. GitHub Repository

GitHub link:

<https://github.com/DariusIsaack/wildlife-conservation-dbms>

7. Conclusion

The Wildlife Conservation Monitoring Database Management System successfully demonstrates the implementation of a relational database integrated with a web-based GUI. The project achieved the objectives of wildlife data management, observation tracking, analytical reporting, and frontend-backend integration.